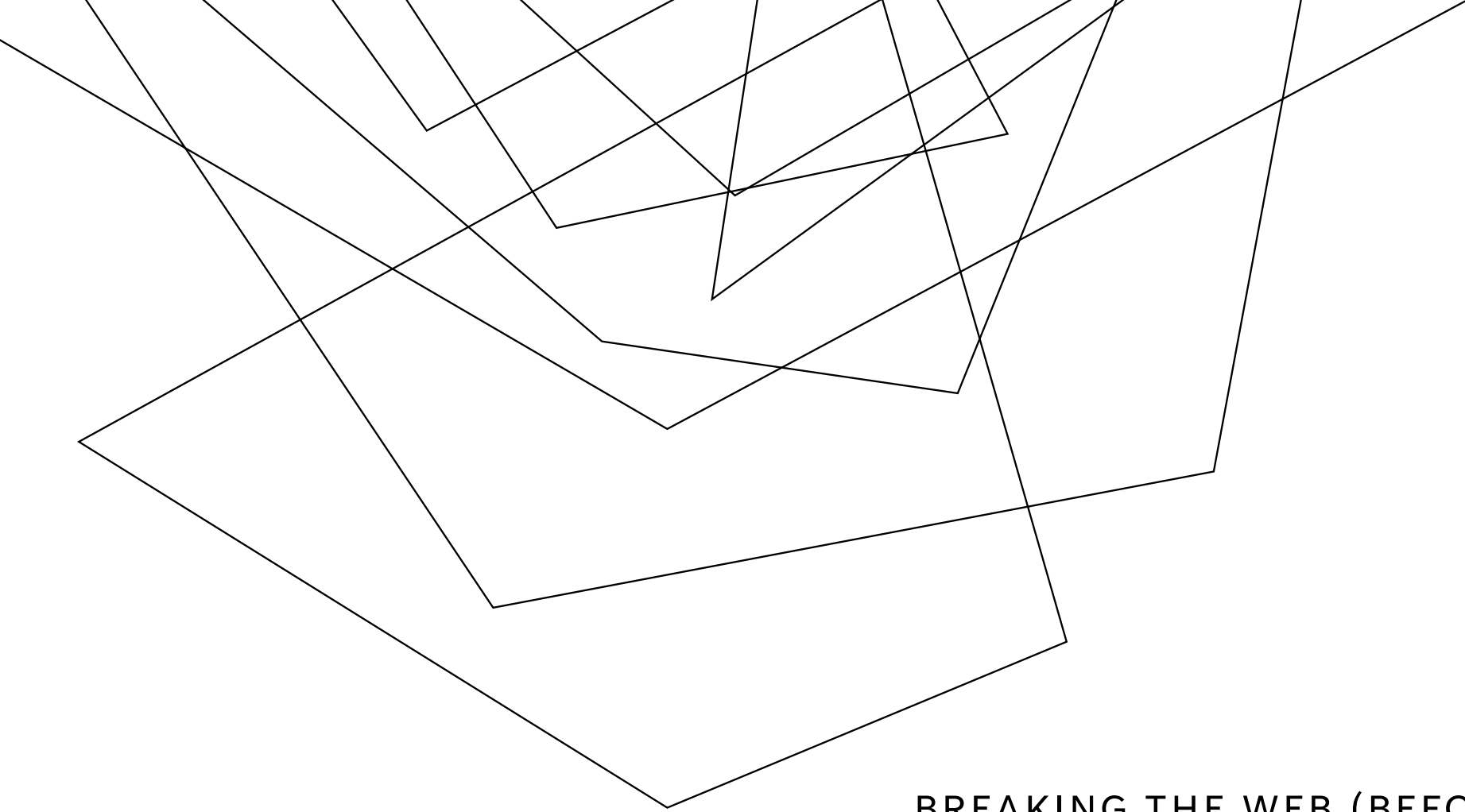




BREAKING THE WEB (BEFORE YOUR USERS DO)

Automated browser testing with Python & Splinter

WHY WRITE BROWSER TESTS?

- Verify user workflows work as expected
- Catch UI bugs
- Ensure your application handles errors and edge-cases gracefully
- Faster, easier, and more reliable than manual testing alone

A series of thin, black, intersecting lines in the top-left corner of the slide, creating a geometric pattern.

WHAT IS SPLINTER?

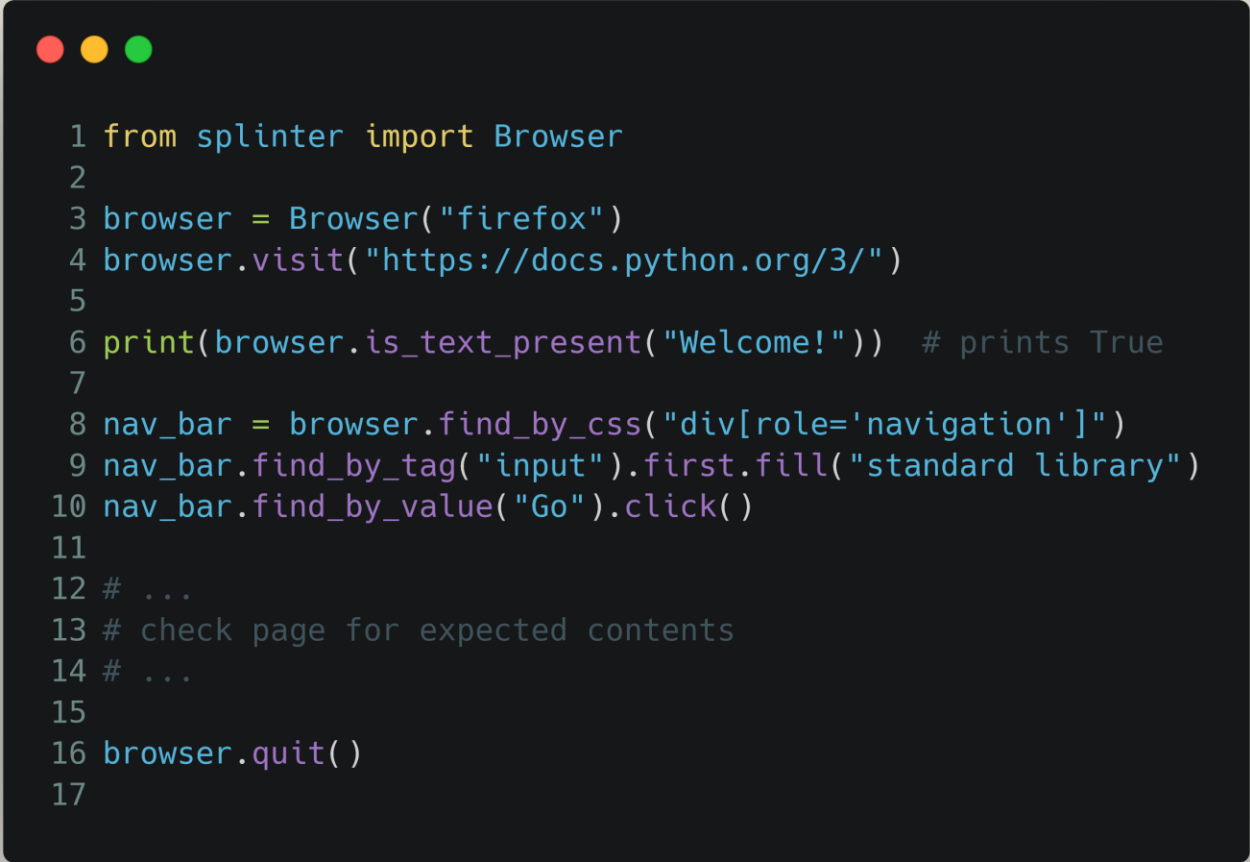
Splinter is a Python package that gives programmers a Pythonic wrapper for the Selenium browser automation toolkit.

The package provides helper functionality for interacting with, and verifying the contents of, web pages.

REQUIREMENTS FOR TESTING WITH SPLINTER

- Python 3.8+
- WebDriver application for your browser
- Splinter and Selenium Python packages
- Basic knowledge of HTML/CSS
- Understanding of automated testing

HOW IT WORKS



```
1 from splinter import Browser
2
3 browser = Browser("firefox")
4 browser.visit("https://docs.python.org/3/")
5
6 print(browser.is_text_present("Welcome!")) # prints True
7
8 nav_bar = browser.find_by_css("div[role='navigation']")
9 nav_bar.find_by_tag("input").first.fill("standard library")
10 nav_bar.find_by_value("Go").click()
11
12 # ...
13 # check page for expected contents
14 # ...
15
16 browser.quit()
17
```

INSTALLATION AND SETUP

1. Create Python virtual environment
2. Enter the virtual environment
3. Install dependencies

Linux & macOS

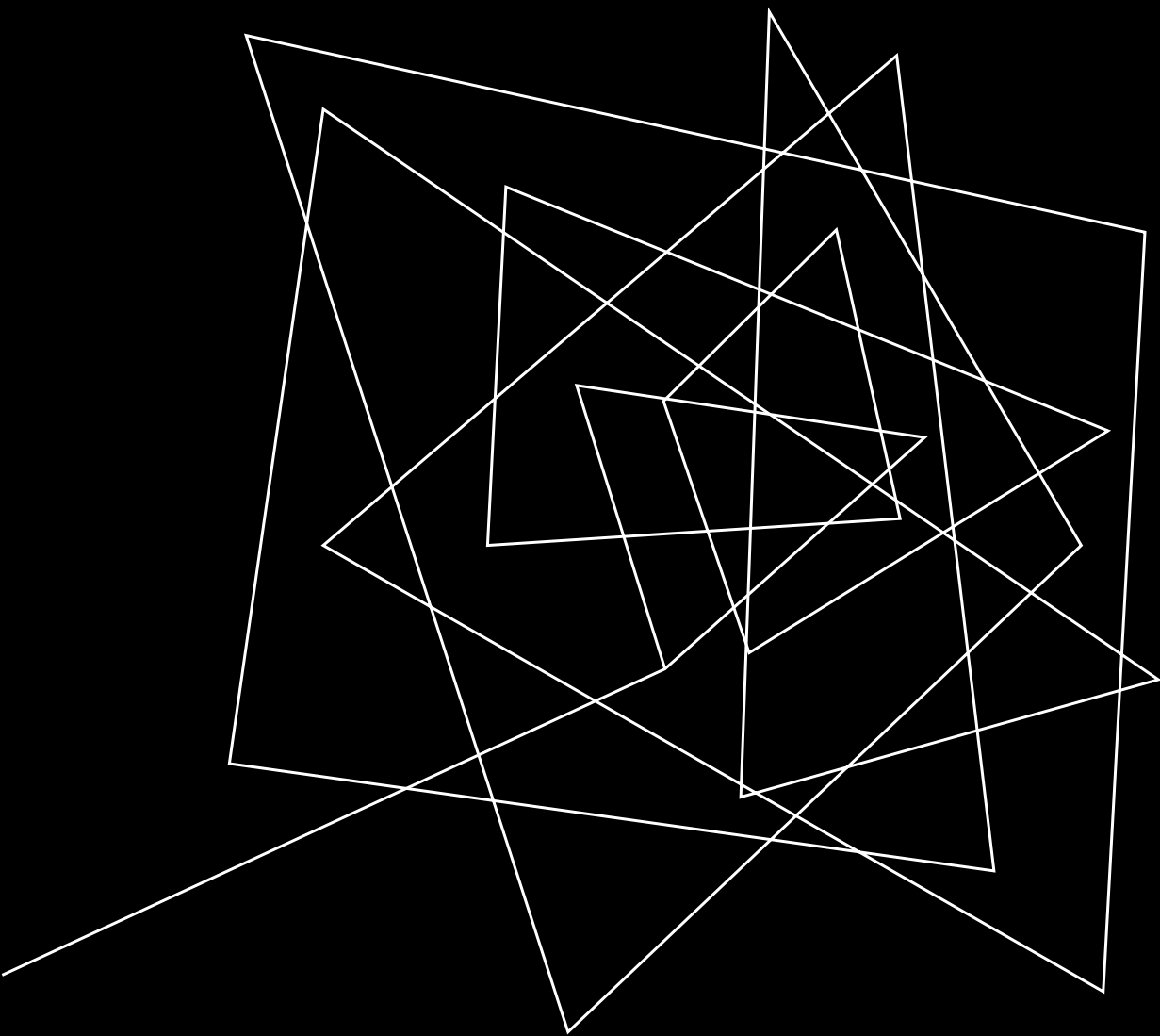


```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install splinter selenium
```

Windows



```
1 py -m venv .venv
2 .venv\Scripts\Activate.bat
3 pip install splinter selenium
```



SPLINTER'S APIs

THE BROWSER API

A small collection of the Browser object's functionality

```
1 # Retrieving elements
2 input_box = browser.find_by_tag("input")
3 number_box = browser.find_by_css("input.quantity")
4 button = browser.find_by_text("Submit")
5
6
7 # Handling alerts, confirmations, and prompts
8 with browser.get_alert(wait_time=5) as alert:
9     print(alert.text)
10    alert.accept()
11
12
13 # Bulk form-filling
14 fields = {
15     "field1": "value1",
16     "field2": "value2",
17     # ...
18 }
19 browser.fill_form(fields, form_id="contact_form")
20
21 # Getting the raw HTML or page text
22 browser.html
23 browser.text
```

THE ELEMENT API

Some functionality for interacting with elements on the page



```
1 # Interacting with elements
2 input_box.type("Kid's bikes", slowly=True)
3 number_box.fill(5)
4 button.click()
5 input_box.clear()
6
7 checkbox.check()
8 checkbox.uncheck()
9
10 # check element visibility, accepts optional wait_time argument
11 button.is_visible()
12 button.is_not_visible()
```

TIME FOR THE LIVE DEMO

Demo Source Code can be found here:

